

Introdução ao JavaScript

Desenvolvimento Web 1



Introdução ao JavaScript © 2026 por Allyson Souza está licenciado sob a **CC BY-NC 4.0**. Para ver uma cópia da licença, visite <https://creativecommons.org/licenses/by-nc/4.0/>

Introdução

Breve contexto da linguagem JavaScript

O quê é JavaScript?

JavaScript® (às vezes abreviado para JS) foi criado para tornar as páginas da web dinâmicas, responderem às interações do usuário.

É uma linguagem interpretada, baseada em objetos, com funções de primeira classe, famosa por ser a linguagem de scripts para páginas web, mas usada também em vários outros ambientes para além dos navegadores, tais como node.js, Apache CouchDB e Adobe Acrobat.

Como executar programas JavaScript?

- Seus programas nesta linguagem são chamados de scripts. Eles podem ser escritos diretamente no HTML de uma página da web e executados automaticamente conforme a página carrega.
- Os scripts são fornecidos e executados como texto simples. Eles não precisam de preparação ou compilação especial para serem executados.
- Nesse aspecto, o **JavaScript** é muito diferente de outra linguagem chamada **Java**.

Características da linguagem

1. Interpretada
2. Tipagem dinâmica e fraca
3. Multi-paradigma: Imperativa; Orientação à objetos; Funcional;
4. Baseada em objetos com protótipos

Introdução ao JavaScript



Por que a linguagem se chama JavaScript?

Quando o JavaScript foi criado, ele tinha inicialmente outro nome: “LiveScript”. Mas o Java era muito popular naquela época, então decidiu-se que posicionar uma nova linguagem como o “irmão mais novo” do Java seria vantajoso.

O JavaScript é uma linguagem totalmente independente, com sua própria especificação chamada **ECMAScript**, e não tem nenhuma relação com o Java para além dessa curiosidade e similaridades léxicas e sintáticas.

Tipagem dinâmica e fraca

Em linguagens com tipagem dinâmica e fraca, o tipo de dado de uma variável não precisa ser informado, pois é determinado (ou modificado) em tempo de execução. Além disso, são linguagens que permitem conversões implícitas (automáticas) de tipos.

Por exemplo:

```
console.log(10 + "10"); // 1010
```

Ambiente de execução

Hoje, o JavaScript pode ser executado não apenas no navegador, mas também em servidores ou, na verdade, em qualquer dispositivo que possua um programa especial chamado de ***JavaScript Engine***.

O navegador possui um motor embutido, chamado de “máquina virtual de JavaScript”.

Motores diferentes têm nomes diferentes. Os “Três Grandes” motores modernos são:

- **V8** – nos navegadores Chrome, Edge, Opera e Brave, e também no Node e Deno
- **SpiderMonkey** – no Firefox.
- **JavaScriptCore** – no Safari e no Bun.

Introdução ao JavaScript



Como funcionam os motores?

Os motores são sistemas complexos, mas os princípios básicos são simples.

1. O motor (embutido, se for um navegador) lê (“analisa”) o script.
2. Em seguida, ele converte (“compila”) o script em código de máquina.
3. E então o código de máquina é executado.

O motor aplica otimizações em cada etapa do processo.

O que JavaScript pode fazer?

O JavaScript moderno é uma linguagem de programação "segura". Ele não oferece acesso de baixo nível à memória (RAM) ou à CPU, pois foi criado inicialmente para navegadores, que não exigem isso.

Os recursos do JavaScript dependem muito do ambiente em que está sendo executado.

JavaScript no navegador

O JavaScript no navegador pode realizar todas as tarefas relacionadas à manipulação de páginas da web, à interação com o usuário e ao servidor web. Por exemplo, o JavaScript no navegador é capaz de:

- **Conteúdo:** adicionar novo código HTML à página, alterar o conteúdo existente e modificar estilos.
- **Interatividade:** reagir às ações do usuário, ser executado ao clicar com o mouse, ao mover o ponteiro ou ao pressionar teclas. Obter e definir cookies, fazer perguntas ao visitante, exibir mensagens.
- **Processamento de dados:** enviar solicitações pela rede para servidores remotos, baixar e enviar arquivos, realizar cálculos, processar texto e controlar sua exibição aos usuários.

JavaScript no navegador: o que não é permitido

As capacidades do JavaScript no navegador são limitadas para proteger a segurança do usuário. O objetivo é impedir que uma página da web maliciosa acesse informações privadas ou prejudique os dados do usuário.

- O JavaScript em uma página da web não pode ler/gravar arquivos arbitrários no disco rígido, copiá-los ou executar programas. Ele não tem acesso direto às funções do sistema operacional.

JavaScript no navegador: o que não é permitido

#1 Leitura e escrita de arquivos

O JavaScript em uma página da web não pode ler/gravar arquivos arbitrários no disco rígido, copiá-los ou executar programas. Ele não tem acesso direto às funções do sistema operacional.

JavaScript no navegador: o que não é permitido

#2 "Same Origin Policy" - Política da mesma origem

Guias/janelas diferentes geralmente não têm conhecimento umas das outras.

Às vezes têm, por exemplo, quando uma janela usa JavaScript para abrir outra. Mas mesmo nesse caso, o JavaScript de uma página não pode acessar a outra página se elas forem de sites diferentes (de um domínio, protocolo ou porta diferentes).

Isso é chamado de “Política de Mesma Origem”. Para contornar isso, ambas as páginas devem concordar com a troca de dados e conter um código JavaScript especial que faça essa gestão.

O que faz do JavaScript uma linguagem única?

- Integração total com HTML e CSS
- Sintaxe simples e versátil
- Suportada por todos os principais navegadores e habilitada por padrão

Linguagens que transpilam para JavaScript

Diferentes pessoas e projetos possuem diferentes necessidades, então o JavaScript não agrada a todos. Por isso hoje existem uma série de linguagens que são *transpiladas*, ou seja, possuem código semelhante, mas não igual, e que compilam para JavaScript.

Algumas delas: **TypeScript**, **ClojureScript**, **Kotlin**

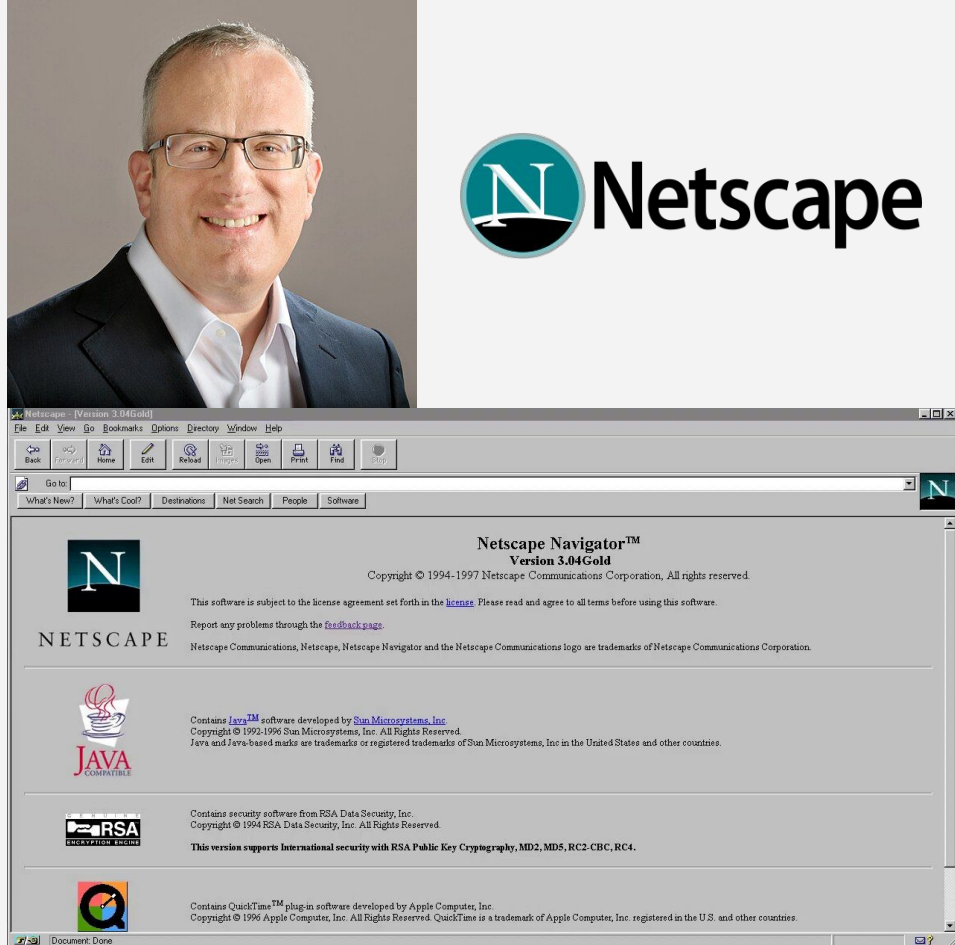
A guerra dos navegadores

A evolução da linguagem em meio à guerra dos navegadores

História da linguagem

Criada por [Brendan Eich](#), em parceria da [Sun Microsystems](#) (atual Oracle) com a [Netscape](#) (subsidiária da AOL - America Online), em 1995, para rodar no navegador Netscape, e foi lançado em março de 1996.

Curiosidade: Brendan criou a primeira versão da linguagem em apenas 10 dias.



História da linguagem

- Em outubro de **1996**, as especificações da linguagem foram submetidas à apreciação da **ECMA International** ([European Computer Manufacturers Association](#)), associação fundada em 1961 dedicada à padronização de sistemas de informação.
- Em junho de 1997, foi publicada a ECMAScript 1, primeira versão padronizada da JavaScript.
- A partir disso, o nome oficial da linguagem passou a ser ECMAScript, sendo a versão mais recente a ECMAScript 16, publicada em junho de 2025.



Anos 90

Em 1995, o **Netscape dominava o mercado de navegadores**. A Microsoft, já uma gigante da informática naquela época, concentrava seus esforços em outras frentes, em outros produtos, acreditavam que precisam dominar a rede, a infraestrutura, não estavam muito atentos aos navegadores.

O sucesso do Netscape chamou a atenção da Microsoft, que então realizou o lançamento de seu navegador, o Internet Explorer 1.0, como parte integrante do Windows 95 Plus Pack, em agosto de 1995.



Abstract

— Friends & Family

Source: U.S. Census Bureau, 1997.

Source: U.S. Census Bureau. "U.S. Births by Race and Hispanic or Latino Ethnicity, 1990-2000." <http://www.census.gov/hhes/immigration/births.html>. Accessed 10/10/2006.

How much do you know?

Source: *Author's calculations*

© 2000 Blackwell Science Ltd

Browser Wars

O termo é atribuído ao jornalismo, no ano de 1996, quando a Microsoft decide competir com a Netscape pelo mercado de navegadores web.

Há referências e subdivisões quanto a estas disputas, por vezes chamadas de "first browser war", "second browser war", mas o fato é que se trata de uma disputa comercial por uma fatia de mercado de uma plataforma que à época, era inovadora e o potencial de seu domínio ainda se delineava.



shoze.blogspot.com

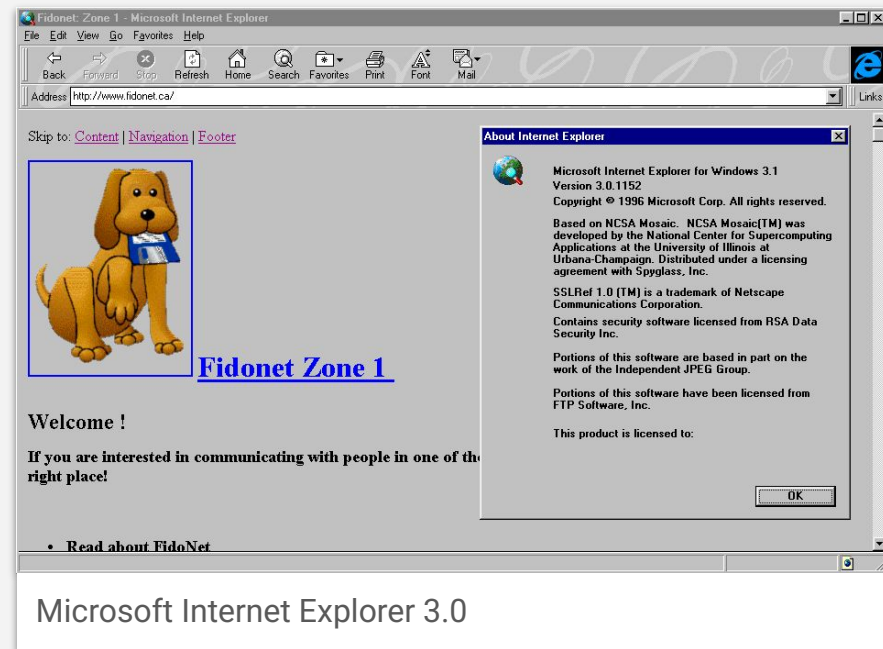
Funcionalidades e mais funcionalidades

Este foi um período de rápidos lançamentos, em que ambas as empresas lançavam novas funcionalidades, a fim de atrair novos consumidores de um mercado emergente, e correções de falhas ficavam em segundo plano.

Em algumas versões, o Internet Explorer começou a de fato fazer frente ao Netscape, a partir da 3.0, em 1996, mais precisamente.

Ainda assim, até 1997, o Netscape tinha **72% do mercado** de navegadores web.

Curiosidade: havia um ditado dentro da Microsoft que todos os seus produtos apenas começavam a dar certo a partir da versão 3.



O que estava em disputa?

Havia muita coisa em disputa entre as empresas neste momento.

Um navegador representava o potencial de se fazer muito dinheiro: mecanismos de busca queriam ser a ferramenta padrão dos navegadores; empresas queriam estar presentes na lista padrão de favoritos; os navegadores seriam (se não o são, ainda), grandes portões para a informação que estava, e estaria, disponível na World Wide Web.



Vantagens da Microsoft

- **Recursos:** Microsoft era uma empresa com mais recursos financeiros que a Netscape. O que a possibilitou, nos EUA, a parcerias com a
- **Consolidação geral na TI:** Netscape tinha um produto: seu navegador. Microsoft toda uma cadeia de aplicações, serviços e infraestrutura;
- **Windows:** a Microsoft tinha um praticamente monopólio dos sistemas operacionais, possibilitando a entrega imediata de seu navegador, Internet Explorer, junto com as instalações do Windows;

A competitividade entre as empresas

"As coisas saíram um pouco do controle em 1997, quando a Microsoft lançou o Internet Explorer 4.0. A equipe construiu uma letra "e" gigante e colocou sorrateiramente no gramado da sede da Netscape. A equipe da Netscape prontamente derrubou o "e" gigante e colocou seu próprio mascote dinossauro Mozilla em cima."



Equipe do Internet Explorer em frente ao edifício da Netscape.

A competitividade entre as empresas

"As coisas saíram um pouco do controle em 1997, quando a Microsoft lançou o Internet Explorer 4.0. A equipe construiu uma letra "e" gigante e colocou sorrateiramente no gramado da sede da Netscape. A equipe da Netscape prontamente derrubou o "e" gigante e colocou seu próprio mascote dinossauro Mozilla em cima."



Mascote da Netscape colocado sobre o "e" do Internet Explorer.

Ação antitruste contra a Microsoft

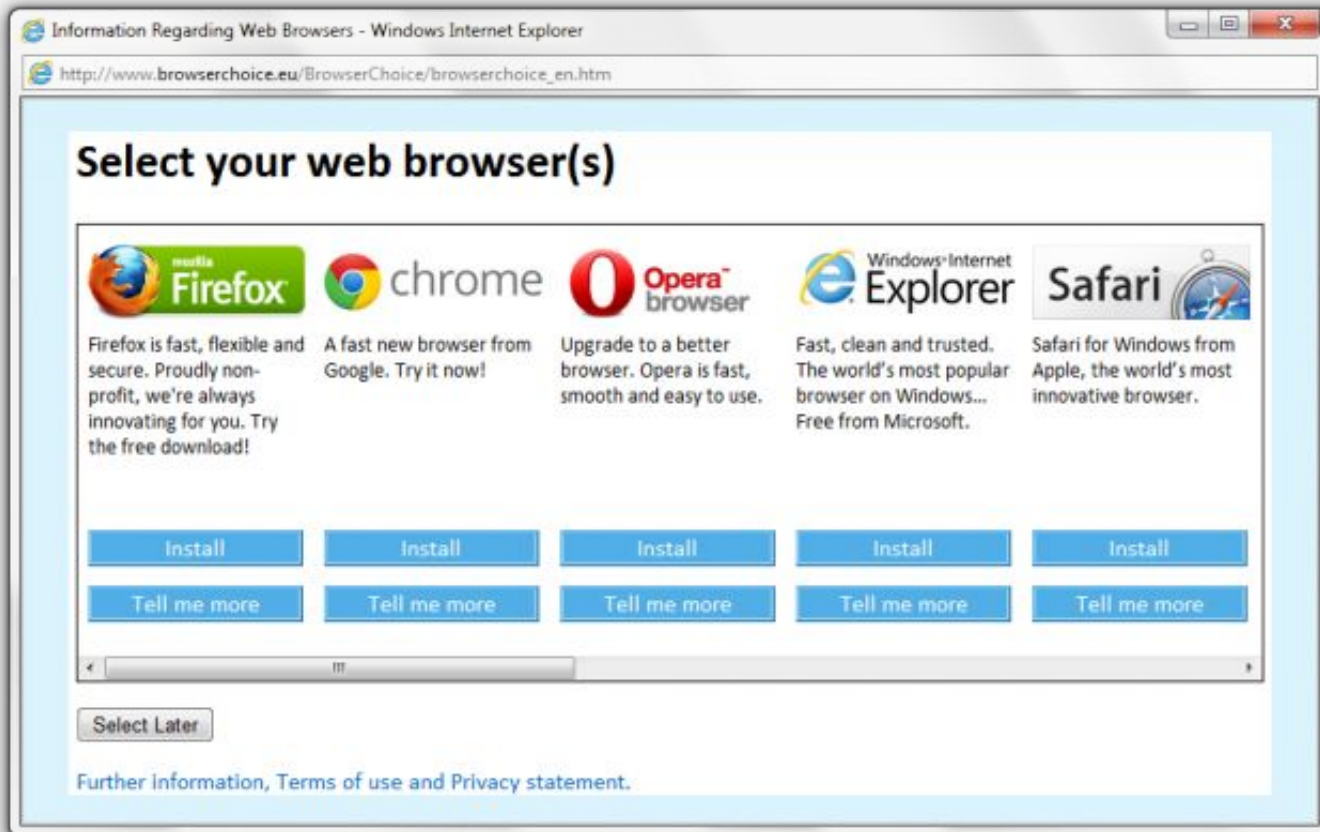
Um caso alegando que a Microsoft utilizou de sua posição monopolista no mercado para impedir a concorrência e beneficiar seu produto, impedir que outros navegadores web tivessem espaço, foi aberto. O processo começou com a Netscape, mas teve outros desdobramentos e chamou atenção do Departamento de Justiça.

O processo durou alguns anos, mas a Microsoft foi condenada a se dividir em duas empresas.





O presidente da Microsoft, Bill Gates (à esquerda), presta depoimento no Capitólio, na terça-feira, 3 de março de 1998. (Crédito: Ken Cedeno/foto de arquivo da AP)



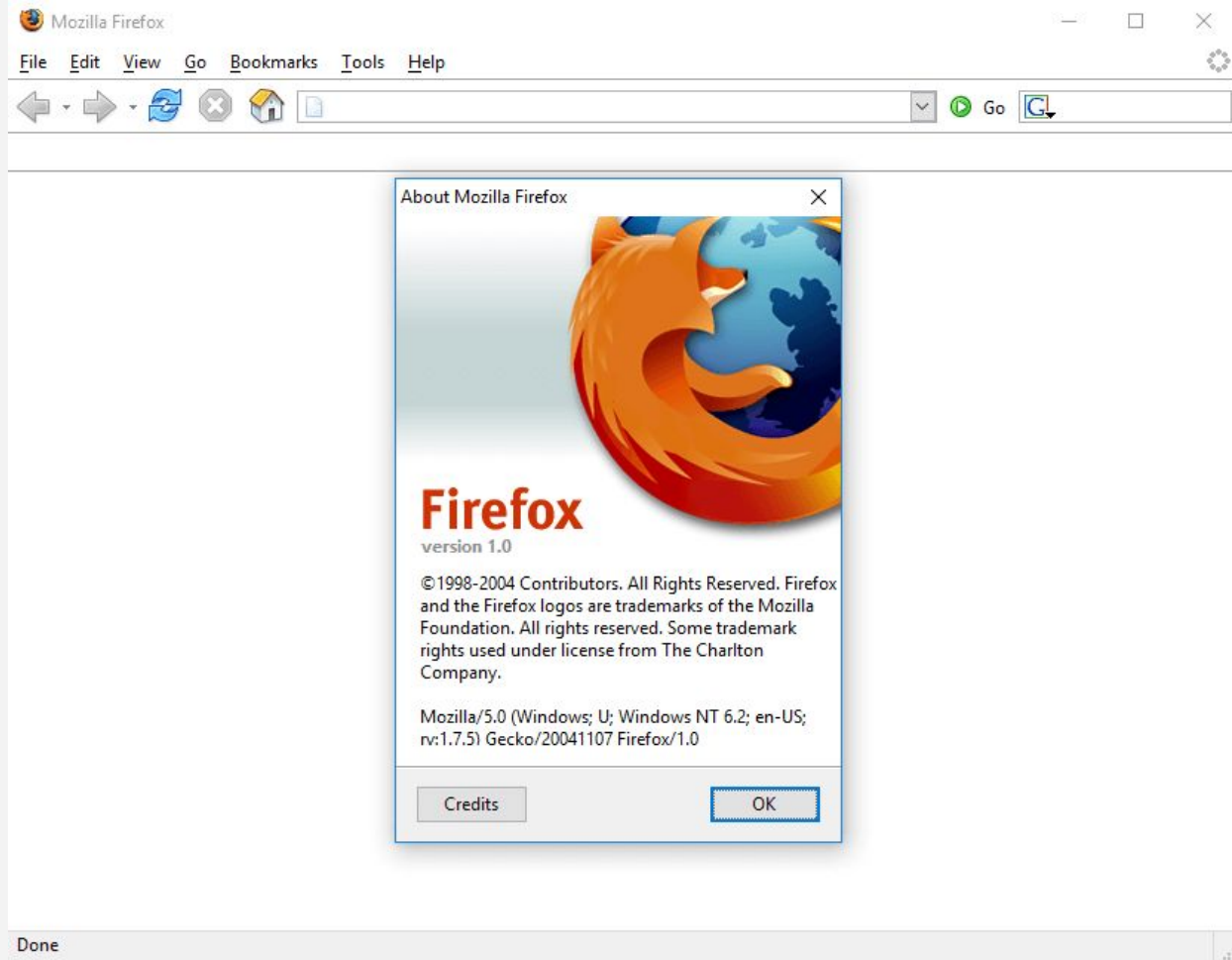
Internet Explorer, 2010, forçados pela União Europeia a exibir uma página com navegadores concorrentes, por 5 anos.

Anos 2000

A Netscape decidiu tornar o seu navegador em um software de código aberto e foi fundada a [Mozilla Foundation](#) e o navegador Firefox, que serviu de principal alternativa ao Internet Explorer durante os anos de 2000-2008, até o lançamento do Chrome, pela Google.

Com um novo motor de renderização e uma engenharia que oferecia melhor performance, gerenciamento de memória, investimento pesado em usabilidade e funcionalidades exclusivas, o **Chrome se tornou e se mantém líder no mercado.**

A Microsoft descontinuou o Internet Explorer após uma perda gradual de usuários, lançando seu novo navegador, Edge, baseado no motor V8, o mesmo do Chrome.



Firefox 1.0

JavaScript

Inserção de scripts na página

Inserindo script: método interno

Método interno: inserir a tag `<script></script>` e escrever nosso código dentro dela. Todo código escrito dentro desta tag será interpretado como JavaScript em nosso navegador.

```
<!doctype html>

<html lang="pt-br">

    <head>

        <title>Script Interno</title>

        <script>

            console.log("Este é um script interno!");

        </script>

    </head>

    <body>

        </body>

</html>
```

Inserindo script: método inline

Nesse método o código JavaScript é inserido diretamente em um atributo de um elemento HTML, em geral disparado na ocorrência de um evento.

```
<!doctype html>

<html lang="pt-br">

  <head>

    <meta charset="UTF-8" />

    <title>Script Inline</title>

  </head>

  <body>

    <button onclick="alert('Clicado!')">

      Clique aqui

    </button>

  </body>

</html>
```

Inserindo script: método externo

Nesse método utilizamos a tag script, mas com o atributo src, referenciamos um arquivo .js externo que será carregado e executado.

Permite a reutilização de scripts, ou mesmo o carregamento de scripts que não estão hospedados no mesmo servidor.

```
<!doctype html>

<html lang="pt-br">

  <head>

    <meta charset="UTF-8" />

    <title>Script Interno</title>

    <script src="meu_script.js"></script>

  </head>

  <body>

    </body>

</html>
```

JavaScript

Sintaxe da linguagem

Maiúsculas e minúsculas

JavaScript é uma linguagem que distingue maiúsculas e minúsculas. Isso significa que as palavras chave da linguagem, variáveis, nomes de funções e outros identificadores devem sempre ser digitados de maneira consistente quanto a este aspecto.

A palavra chave **while**, por exemplo, deve ser digitada **while**, não **While** ou **WHILE**.

O mesmo é válido para nomes de variáveis. Um variável chamada **online** precisa ser sempre referenciada dessa forma, **Online** ou **ONLINE** serão interpretados como variáveis diferentes.

Espaços

O JavaScript ignora os espaços que aparecem entre os tokens nos programas. Na maioria dos casos, o JavaScript também ignora as quebras de linha na maioria dos casos.

Como é possível usar espaços e quebras de linha livremente em seus programas, você pode formatar e recuar seus programas de maneira organizada e consistente, tornando o código fácil de ler e compreender.

Comentários

O JavaScript suporta dois tipos de comentários.

1. Qualquer texto entre `//` e o final de uma linha é tratado como um comentário e ignorado pelo JavaScript.
2. Qualquer texto entre os caracteres `/*` e `*/` também é tratado como um comentário; esses comentários podem se estender por várias linhas.

```
// Este é um comentário de uma linha.
```

```
/* Este também é um comentário */ // e aqui está outro comentário.
```

```
/*  
 * Este é um comentário de várias linhas. Os caracteres  
 * extras no início de  
 * cada linha não são obrigatórios na sintaxe; eles só  
 * ficam legais!  
 */
```

Literais

Literal é qualquer valor que aparece diretamente no programa. Os valores ao lado são todos literais.

`12` // 0 número doze

`1.2` // 0 número um ponto dois

`"olá mundo"` // Uma sequência de texto

`'Oi'` // Outra sequência de texto

`true` // Um valor booleano

`false` // 0 outro valor booleano

`null` // Ausência de um objeto

Identificadores e palavras reservadas

Um identificador é simplesmente um nome.

No JavaScript, os identificadores são usados para nomear **constantes**, **variáveis**, **propriedades**, **funções** e **classes**.

Um identificador em JavaScript **deve** começar com uma letra, um sublinhado (`_`) ou um cifrão (`$`). Os caracteres seguintes podem ser letras, dígitos, sublinhados ou cifrões.

(Não é permitido usar dígitos como primeiro caractere, para que o JavaScript possa distinguir facilmente identificadores de números)

```
// Identificadores válidos
```

```
i
```

```
minha_variável
```

```
v13
```

```
__carro
```

```
$string
```

```
// Identificadores inválidos
```

```
1pessoa
```

```
#carro
```

```
lista*de*compras
```

```
>alvo<
```

Palavras reservadas

As palavras reservadas são parte da linguagem. Elas não devem ser usadas como nomes de variáveis, constantes, funções ou classes.

as	const	export	get	null	target	void
async	continue	extends	if	of	this	while
await	debugger	false	import	return	throw	with
break	default	finally	in	set	true	yield
case	delete	for	instanceof	static	try	
catch	do	from	let	super	typeof	
class	else	function	new	switch	var	

Palavras reservadas

Algumas palavras reservadas ainda não são usadas pela linguagem, mas podem vir a ser, por isso também são restritas.

enum	implements	interface	package	private	protected	public
------	------------	-----------	---------	---------	-----------	--------

Ponto e vírgula

Como muitas outras linguagens, JavaScript também utiliza ponto e vírgula para separar instruções. Isso é importante para tornar o seu código compreensível.

Sem o ponto e vírgula, o final de uma instrução pode ser confundido com o início de outra.

// Ponto e vírgula opcional

```
a = 3;
```

```
b = 4;
```

```
c = 5;
```

// Ponto e vírgula obrigatório

```
a = 3; b = 4; c = 5;
```

JavaScript

Tipos, valores e variáveis

Declaração de variáveis

Para declararmos variáveis no JavaScript, usamos a palavra chave **let** seguida do nome que queremos para nossa variável (obedecendo a regra de nomeação de identificadores da linguagem).

```
// Declaração da variável
```

```
let mensagem;
```

```
// Atribuição de um valor à variável
```

```
mensagem = 'Olá!';
```

```
// Usando o valor da variável
```

```
alert(mensagem);
```

Declaração de variáveis: múltiplas por linha

Podemos declarar múltiplas variáveis por linha, separando seus identificadores por vírgula, ou mesmo usando uma instrução `let`, separando por vírgulas e quebrando linhas

Não é muito recomendado, dado o prejuízo quanto à legibilidade do código.

```
let usuario = 'John', idade = 25;
```

```
let usuario = 'John',  
    idade = 25;
```

Tipos de dados

Existem alguns tipos de dados que podemos armazenar em JavaScript.

O JavaScript possui ***tipagem dinâmica***, o que significa que você não precisa dizer ao navegador qual é o tipo do dado; ele descobre sozinho em tempo de execução, ou seja, enquanto o código é interpretado por ele - diferente de algumas outras linguagens.

Strings

Representa dados textuais, como nomes, códigos alfanuméricos, mensagens, etc.

Uma String deve sempre estar envolvida por aspas simples (' '), aspas duplas (" ") ou crases (` `).

```
let nome = "José"
```

```
let rua = "R. das Flores, 123"
```

```
let cpf = "222.222.222-22"
```

Strings

As crases tem uma “funcionalidade estendida”.

Elas nos permitem incorporar variáveis e expressões em uma string, colocando-as entre `${...}`, por exemplo:

```
let name = "Jessica";
```

```
// Usando crases para incorporar a  
variável
```

```
alert( `Olá, ${name}!` );
```

```
// embed an expression
```

```
alert( `the result is ${1 + 2}` ); //  
the result is 3
```

Números

Representa dados numéricos.

Diferente de outras linguagens (que separam inteiros e decimais), o JavaScript usa o tipo Number para tudo: números inteiros (10), decimais (3.14) ou negativos (-5).

```
let idade = 27
```

```
let saldo = 110.57
```

```
let temperatura = -2
```

Números: valores numéricos especiais

Para além dos números, existem os chamados valores numéricos especiais, que são:

- **Infinity**
- **-Infinity**
- **NaN**

Os valores numéricos especiais pertencem formalmente ao tipo “número”. É claro que não são números no sentido comum da palavra.

```
// Divisão por zero  
alert( 1 / 0 ); // Infinity
```

```
// Multiplicação com string  
alert( "A" * 2 ); // NaN
```

Booleano

Pode ter apenas um de dois valores: **true** (verdadeiro) ou **false** (falso).

É fundamental para tomar decisões lógicas no código (estruturas como o if).

```
let maiorDeIdade = true
```

```
let matriculado = true
```

```
let ligado = false
```

Undefined

É o valor padrão atribuído pelo JavaScript a uma variável que foi declarada, mas que ainda não recebeu nenhum valor.

Todas as variáveis ao lado, a princípio, terão armazenadas o valor **undefined** (indefinido).

```
let nome
```

```
let idade
```

```
let endereco
```

Null

É a ausência *intencional* de qualquer valor.

Diferente do undefined, o null é atribuído pelo programador quando ele quer dizer explicitamente que "esta variável está vazia".

```
let nome = null
```

```
let idade = null
```

```
let endereco = null
```

Operador typeof

Muitas vezes não sabemos bem, ou iremos querer nos certificar de que uma variável está armazenando um determinado tipo de dados. Para isso, podemos utilizar o operador typeof.

Uso do typeof

```
let nomeDoAluno = "Marcos";  
console.log("Valor:", nomeDoAluno);  
console.log("Tipo:", typeof nomeDoAluno);
```

Declaração com let ou const

Declaração com **let**

No JavaScript atual (ES6 em diante), as variáveis são declaradas com a instrução **let**.

- É uma boa prática de programação atribuir valores ao declarar variáveis.
- Podemos utilizar as variáveis previamente declaradas na atribuição de valor das novas.

```
// Uma variável por linha
```

```
let i;
```

```
let sum;
```

```
// Múltiplas variáveis por linha
```

```
let i, sum;
```

```
// Atribuição de valor ao declarar
```

```
let message = "hello";
```

```
let i = 0, j = 0, k = 0;
```

```
let x = 2, y = x*x;
```

Declaração com **const**

Utilizando a instrução **const**, podemos declarar variáveis que não podem ter seu valor redefinido.

Não podemos declarar variáveis com **const** e depois atribuir valor a elas.

```
const PI = 3.14159;  
  
const MAXIMO_TENTATIVAS_LOGIN = 5;  
  
const API_URL = "https://example.com";  
  
// Inválido  
  
const ACELERACAO;  
  
const MIN_VALUE;
```

Existem duas correntes de pensamento sobre o uso da palavra-chave `const`.

Uma abordagem consiste em usar `const` apenas para valores que são fundamentalmente imutáveis, como as constantes físicas mostradas, ou números de versão de programas, ou sequências de bytes usadas para identificar tipos de arquivos, por exemplo.

Outra abordagem reconhece que muitas das chamadas “variáveis” em nosso programa, na verdade, nunca mudam enquanto o programa está em execução. Nessa abordagem, declaramos tudo com “`const`” e, se percebermos que realmente queremos permitir que o valor varia, alteramos a declaração para “`let`”. Isso pode ajudar a evitar bugs ao impedir alterações acidentais em variáveis que não pretendíamos.

Em uma abordagem, usamos `const` apenas para valores que não devem mudar. Na outra, usamos `const` para qualquer valor que por acaso não mude. Eu prefiro a primeira abordagem no meu próprio código.

David Flanagan, **JavaScript: The Definitive Guide**, p. 72

Interação básica

`alert()` / `prompt()` / `confirm()` / `console`

alert()

A instrução `alert()` exibe uma mensagem em uma janela de diálogo (pop-up) do navegador.

Essa mensagem precisa ser passada entre aspas, dentro dos parênteses.

```
alert("Estou aprendendo JavaScript!");
```

confirm()

A função **confirm** exibe uma janela modal com uma pergunta e dois botões: **OK** e **Cancelar**.

O resultado é **true** se o botão OK for pressionado e **false** caso contrário.

```
let aluno
```

```
aluno = confirm("Você é um aluno?");
```

```
alert( aluno ); // true ou false
```

prompt()

Exibe uma janela modal com uma mensagem de texto, um campo de entrada para o visitante e os botões OK/Cancelar.

título

O texto a ser exibido ao visitante.

padrão

Um segundo parâmetro opcional, que representa o valor inicial do campo de entrada.

```
let idade;
```

```
idade = prompt('Qual sua idade?', 100);
```

```
alert(`Você tem ${age} anos!`);
```

```
// Saída: Você tem __ anos!
```

console.log()

O `console.log()` é a ferramenta padrão de diagnóstico do desenvolvedor.

Ele não exibe nada na tela para o usuário final; em vez disso, ele envia o dado diretamente para o painel de controle interno do navegador (as chamadas DevTools).

```
let nome = "Marcos";  
  
console.log("Valor:", nome);
```

Conversão de tipos

Conversão de tipos

Na maioria das vezes, os operadores e as funções convertem automaticamente os valores que lhes são fornecidos para o tipo correto.

Por exemplo, a função **alert()** converte automaticamente qualquer valor em uma string para exibi-lo.

As operações matemáticas convertem valores em números.

Conversão de tipos

`10 + " objetos" // "10 objetos"` número 10 é convertido em string

`"7" * "4" // => 28`, ambas as strings são convertidas para números

`let n = 1 - "x"; // n == NaN`; string "x" can't convert to a number

`n + " objects" // "NaN objects"`: NaN é convertido para string "NaN"

Tabela de conversões

parte 1

Resume como os valores são convertidos de um tipo para outro no JavaScript.

As entradas em negrito na tabela destacam conversões que podem te surpreender. As células vazias indicam que nenhuma conversão é necessária e, portanto, nenhuma é realizada.

Valor	to String	to Number	to Boolean
undefined	"undefined"	NaN	false
null	"null"	0	false
true	"true"	1	true
false	"false"	0	false
" " (empty string)	""	0	false
"1.2" (nonempty, numeric)	"1.2"	1.2	true
"one" (nonempty, non-numeric)	"one"	NaN	true
0	"0"		false
-0	"0"		false
1 (finito, não-zero)	"1"		true

Tabela de conversões

parte 2

Resume como os valores são convertidos de um tipo para outro no JavaScript.

As entradas em negrito na tabela destacam conversões que podem te surpreender. As células vazias indicam que nenhuma conversão é necessária e, portanto, nenhuma é realizada.

Valor	to String	to Number	to Boolean
Infinity	"Infinity"		true
-Infinity	"-Infinity"		true
NaN	"NaN"		false
{ } qualquer objeto	<i>a detalhar mais adiante</i>	<i>a detalhar mais adiante</i>	true
[] array vazio	""	0	true
[9]	"9"	9	false
['a']	precisa do método join()	NaN	true
function() {}	<i>a detalhar mais adiante</i>	NaN	true

Conversões e igualdades

Conversões e igualdade

O JavaScript possui dois operadores que verificam se dois valores são iguais.

O “operador de igualdade estrita”, `===`, não considera seus operandos iguais se eles não forem do mesmo tipo, e esse é quase sempre o operador correto a ser usado na programação.

Mas, como o JavaScript é tão flexível em relação às conversões de tipo, ele também define o operador `==` com uma definição flexível de igualdade.

Todas as comparações a seguir são verdadeiras, por exemplo:

Conversões e igualdade

`null == undefined` // => true: estes dois valores são tratados como iguais

`"0" == 0` // => true: String converte para número antes da comparação

`0 == false` // => true: Boolean converte para número antes da comparação

`"0" == false` // => true: Ambos operandos são convertidos para 0 antes da comparação

Conversões explícitas

Conversão de strings

A conversão de string ocorre quando precisamos da forma de string de um valor.

Por exemplo, a função **alert(valor)** faz isso para exibir o valor.

Também podemos chamar a função **String(valor)** para converter um valor em uma string:

```
let valor = true;
```

```
alert(typeof valor); // boolean
```

```
valor = String(valor); // agora value  
is a string "true"
```

```
alert(typeof value); // string
```

Conversão de números

A conversão numérica em funções e expressões matemáticas ocorre automaticamente.

Por exemplo, quando a divisão / é aplicada a valores que não são números:

```
alert( "6" / "2" );  
// Saída: 3, as strings são  
// automaticamente convertidas em  
// números
```

Conversão de números

Podemos usar a função **Number(valor)** para converter explicitamente um valor em um número:

```
let palavra = "123";  
alert(typeof palavra); // string  
  
let num = Number(palavra);  
// transforma no número 123  
  
alert(typeof num); // number
```

Conversão de números

Podemos usar a função **Number(valor)** para converter explicitamente um valor em um número:

```
let palavra = "123";  
alert(typeof palavra); // string
```

```
let num = Number(palavra);  
// transforma no número 123
```

```
alert(typeof num); // number
```

Operadores aritméticos

Operadores aritméticos

```
console.log(5 + 7)
```

```
console.log(10 - 4)
```

```
console.log(2 * 8)
```

```
console.log(34 / 17)
```

```
console.log(3 ^ 2)
```

Conditionais

`if / else / else if`

Estrutura da instrução if...else

Veja a estrutura visual da instrução se...senao ao lado. Observe que os caracteres de chaves { } são utilizados para delimitar os blocos de código em Portugol (e em muitas outras linguagens de programação, mas não em todas)

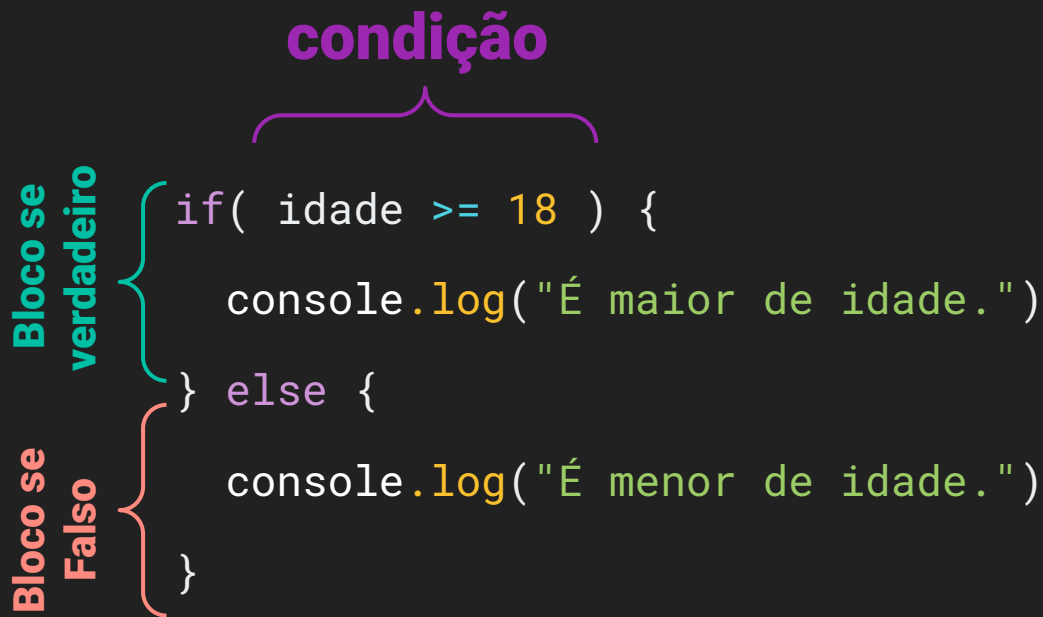


Tabela de operadores relacionais

Operador	Significado	Exemplo	Resultado
<code>===</code>	Igual e do mesmo tipo de	<code>5 === "5"</code>	<code>false</code>
<code>==</code>	Igual a	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	Diferente de	<code>5 != 3</code>	<code>true</code>
<code>></code>	Maior que	<code>10 > 20</code>	<code>false</code>
<code><</code>	Menor que	<code>7 < 15</code>	<code>true</code>
<code>>=</code>	Maior ou igual a	<code>10 >= 10</code>	<code>true</code>
<code><=</code>	Menor ou igual a	<code>12 <= 10</code>	<code>false</code>

Encadeamento

Após a verificação de uma condição if, em um bloco else, posso segui-lo de uma nova verificação, antes de executar diretamente o bloco. Esse encadeamento de condições é o que chamamos de **else if** (senão se...)

```
if( idade >= 60 ) {  
    console.log("É idoso.")  
} else if ( idade >= 18 ) {  
    console.log("É maior de idade.")  
} else {  
    console.log("É menor de idade.")  
}
```

Condicionalis compostas

&& / || / !

Tabela de operadores lógicos

Operador	Significado	Exemplo (Portugol)	Resultado do exemplo
&&	Verdadeiro se as duas condições são verdadeiras	5 > 4 && 17 <= 10	falso
 	Verdadeiro se ao menos uma das condições for verdadeira	10 > 11 10 > 5	verdadeiro
!	Inverte o valor lógico de uma condição	! true	falso

Caso

switch ... case

Sintaxe básica

Essa é a sintaxe básica da estrutura switch case

```
switch (expressão) {  
    case valor1:  
        // executado se expressão == valor1  
        break;  
    case valor2:  
        // executado se expressão == valor2  
        break;  
    default:  
        // executado se nenhum case corresponder  
}
```

Exemplo de uso

Queremos testar se a variável cor possui um dos seguintes valores atribuído a ela.

```
let cor = "Amarelo"

switch(cor){
    case "Amarelo":
        console.log("Amarelo: é uma cor primária")
        break;
    case "Azul":
        console.log("Azul: é uma cor primária")
        break;
    default:
        console.log("Não é uma cor primária")
}
```

Laços de repetição

do ... while / while / for

while

do...while

for

Funções

declaração, propriedades e manipulação

O que são funções?

As funções são um dos recursos mais importantes na programação JavaScript.

O conceito de envolver uma parte de um programa em um valor tem muitos usos. Ele nos permite estruturar programas maiores, reduzir repetições, associar nomes a subprogramas e isolar esses subprogramas uns dos outros.

Três formas de criação de funções

O JavaScript possui 3 (três) principais formas de criação de funções, que são:

1. **Declaração de funções** (ou funções nomeadas)
2. **Expressão de funções** (atribuindo à variáveis, constantes e propriedades de objetos)
3. **Funções seta** (*arrow functions*)

Algumas características de funções

Em JavaScript, funções são objetos de primeira classe — podem ser atribuídas a variáveis e passadas como argumento.

Aninhamento: funções podem ser definidas dentro de outras funções, tendo acesso a todas as variáveis do escopo externo.

1. Atribuindo uma função

Uma atribuição de função é uma declaração de uma variável ou constante em que o valor associado a este identificador é uma função.

Por exemplo, este código define a constante `quadrado` para se referir a uma função que produz o quadrado de um determinado número.

```
const quadrado = function (x) {  
    return x * x;  
};
```

// Invocação da função `quadrado`
passando o argumento 12

```
console.log(quadrado(12));  
  
// Saída → 144
```

Constante que irá armazenar a função

Instrução function

Parâmetro

```
1 const quadrado = function(x) {  
2     return x * x;  
3 }
```

Retorno

Estrutura de atribuição de uma função

Parâmetros

Uma função pode ter **vários parâmetros** ou **nenhum parâmetro**.

No exemplo a seguir, fazerBarulho não lista nenhum nome de parâmetro, enquanto soma (que faz a soma de dois números) recebe dois parâmetros:

```
const fazerBarulho = function() {  
  console.log("Vruuum!")  
}
```

```
fazerBarulho()  
// → Vruuum!
```

```
const soma = function(a, b) {  
  let resultado = a + b  
  return resultado  
};
```

```
console.log(soma(23, 5))  
// → 28
```

2. Declaração de função

Existe uma maneira um pouco mais curta de criar uma função.

Quando a palavra-chave ***function*** é usada no início de uma instrução, ela funciona de maneira diferente

```
function quadrado(x) {  
    return x * x;  
}
```

Declaração de funções / função nomeada

```
function areaDoRetangulo(ladoA, ladoB) {  
    let area = ladoA * ladoB  
    return area  
}
```

// Invocação da função com diferentes parâmetros

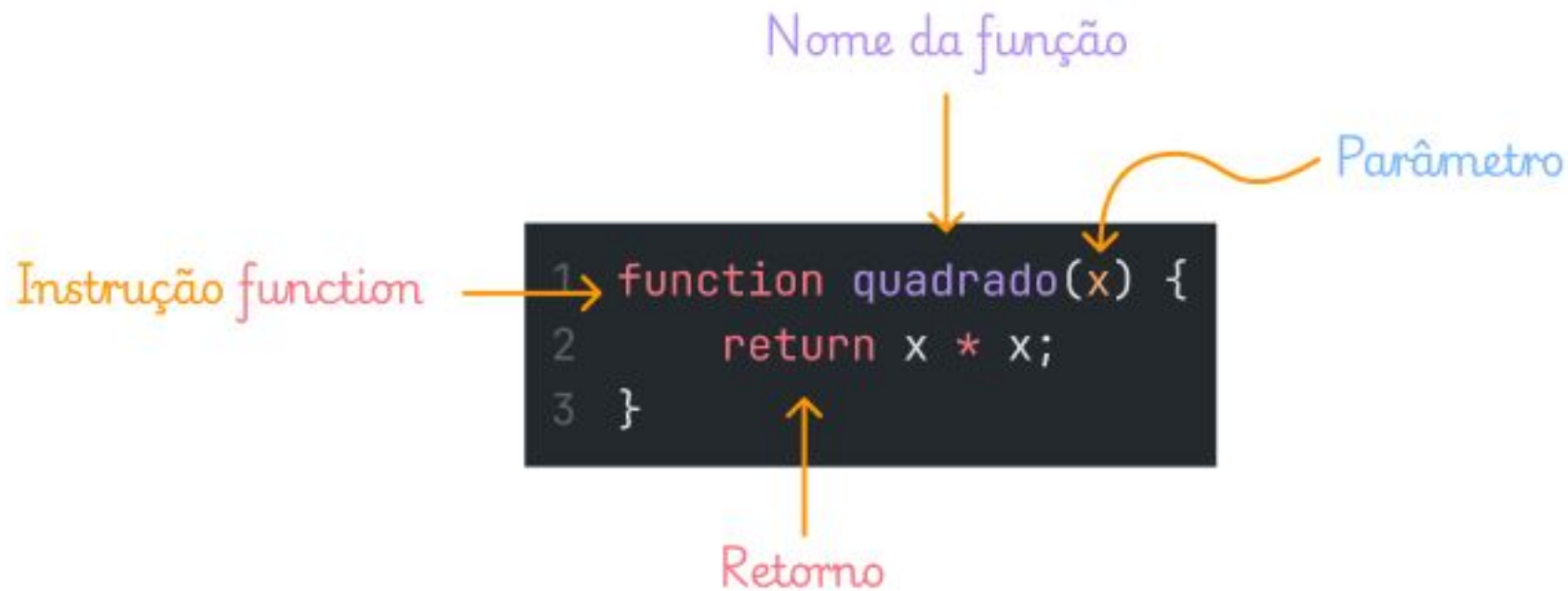
```
areaDoRetangulo(10, 10)
```

```
areaDoRetangulo(2, 4)
```

```
areaDoRetangulo(5, 3)
```

Declaração de funções / função nomeada

```
function areaDoRetangulo(ladoA, ladoB) {  
    let area = ladoA * ladoB  
    return area  
}
```



Estrutura de declaração de uma função

Içamento (*hoisting*)

O código funciona, mesmo que a função seja definida após o código que a invoca.

As declarações de função não fazem parte do fluxo de controle regular de cima para baixo.

Elas são conceitualmente movidas para o topo de seu escopo e podem ser utilizadas por todo o código nesse escopo. Isso às vezes é útil porque oferece a liberdade de ordenar o código da maneira que parecer mais clara, sem se preocupar em definir todas as funções antes de utilizá-las.

```
console.log(  
    "Mensagem do futuro: ",  
    mensagemDoFuturo()  
)  
  
function mensagemDoFuturo() {  
    return "A IA ainda não capina  
lote";  
}
```

3. Funções seta

Introduzidas em 2015 (ES6), as arrow functions oferecem uma sintaxe mais concisa usando o operador `=>`. A seta representa "esta entrada produz este resultado".

```
const media = (n1, n2) => {  
    return (n1 + n2) / 2  
};
```

```
const media = (n1, n2) => { return (n1 + n2) / 2 };
```

1

Parênteses opcionais

Com apenas **um parâmetro**, os parênteses podem ser omitidos.

```
const quadrado = x => x * x;
```

2

Retorno implícito

Se o corpo é uma **única expressão**, as chaves e o `return` são dispensáveis.

```
const quadrado = x => x * x;
```

3

Sem parâmetros

Use um par de parênteses vazio quando não houver parâmetros.

```
const latir = () =>  
  console.log("Roof!");
```

Objetos

declaração, propriedades e manipulação

O que são objetos?

A linguagem JavaScript é projetada com base em um paradigma orientado à objetos.

Objetos

Um objeto é uma **coleção** de **propriedades**.

Uma propriedade é uma associação entre um **nome** (ou *chave*) e um **valor**.

nome : valor

Notação literal

A primeira forma de declaração de objetos é utilizando a instrução **notação literal**.

Basta utilizar as chaves para criar um objeto vazio.

Para definir propriedades de objetos, podemos nomeá-las, tal como variáveis, e então usar `:` para em seguida atribuir seus valores, tal como em variáveis.

```
let meuCarro = {};
```

```
let meuCarro = {  
    fabricante: "Ford",  
    modelo: "Mustang",  
    ano: 1969  
}
```

Notação com construtor

A segunda forma de declaração de objetos é utilizando a instrução `new Object()` para atribuir um novo objeto à uma variável.

Para definir propriedades (nomes e valores), utilizamos o operador de ponto, em seguida do nome da variável que armazena o objeto.

Para atribuir valor à propriedade, utilizamos o operador de atribuição `=`, tal qual usamos normalmente para variáveis.

```
let meuCarro = new Object();  
meuCarro.fabricante = "Ford";  
meuCarro.modelo = "Mustang";  
meuCarro.ano = 1969;
```

Construtor

Construtores de objeto podem utilizar uma função como um modelo para criar objetos.

```
function Car(modelo, ano, cor, marca)
{
    this.modelo = modelo;
    this.ano    = ano;
    this.cor    = cor;
    this.marca  = marca;
    this.rodas  = 4
    this.retrovisor = 2
    this.insulfilm = false
}
```

Usando o Construtor

Após definir o modelo podemos criar instâncias do objeto utilizando a função de construtor.

Utilizamos a palavra-chave `new` seguida por uma chamada de função que irá criar um novo objeto.

```
let carro = new Car(  
    'Gol',  
    2020,  
    'preto',  
    'Volkswagen'  
);
```

Objetos: *this*

Entendendo o `this` em JavaScript

this

A palavra-chave ***this*** é utilizada dentro de funções e objetos.

Sempre que a função é declarada, ela altera o quê ***this*** significa. Ele sempre se refere a um objeto, geralmente o objeto em que a função opera.

Escopo global

Quando uma função é criada no nível superior de um script (isto é, não dentro de outro objeto ou função), então ela está no **escopo global** ou contexto global.

O objeto padrão nesse contexto é o objeto **window**, portanto, quando **this** é utilizada dentro de uma função no contexto global, ela se refere ao objeto **window**.

Métodos

Quando uma função é definida dentro de um objeto, ela torna-se um método. Em um método, **this** refere-se ao objeto que o contém.

```
let car = {  
  modelo: "Hilux",  
  ano: 2021,  
  cor: "Preto",  
  marca: "Toyota",  
  km: 20.000,  
  mostrarKilometragem: function() {  
    return this.km;  
  }  
}
```

Document

Manipulação de conteúdo de páginas web

document

O ***document*** é o objeto global que representa a sua página HTML dentro do navegador. Sempre que você quiser manipular em algo no navegador via JavaScript, você passará por ele.

Ele possui alguns **métodos** que permitem que busquemos por elementos específicos na página para manipulação.

Document: seleccionar elementos

`querySelector / querySelectorAll /
getElementById / getElementsByClassName`

document.querySelector()

```
// Por seletor CSS (mais moderno e flexível)  
const paragrafo = document.querySelector('p');  
const item = document.querySelector('.lista li');
```

`document.getElementById()`

```
// Por ID – retorna apenas o elemento com o atributo  
// id especificado como "titulo" no HTML  
const titulo = document.getElementById('titulo');
```

Document: ler e alterar conteúdo

`textContent` / `innerHTML`

textContent e innerHTML

```
const el = document.querySelector('h1');  
  
// Ler só o texto (sem tags)  
console.log(el.textContent);  
  
// Alterar o texto  
el.textContent = 'Olá, mundo!';  
  
// innerHTML lê/escreve HTML de verdade  
el.innerHTML = 'Olá, <strong>mundo!</strong>';
```

Document: alterar estilos

`style/` `classList`

textContent e innerHTML

```
const caixa = document.querySelector('.caixa');
```

```
// Propriedades CSS em camelCase
```

```
caixa.style.color = 'red';
```

```
caixa.style.backgroundColor = '#f0f0f0';
```

```
caixa.style.fontSize = '20px';
```

```
// Ou adicione/remova classes CSS
```

```
caixa.classList.add('destaque');
```

```
caixa.classList.remove('oculto');
```

```
caixa.classList.toggle('ativo');
```

Document: criar elementos

`createElement` / `appendChild`

createElement / appendChild

```
// 1. Cria o elemento
const paragrafo = document.createElement('p');

// 2. Define o conteúdo
paragrafo.textContent = 'Sou novo aqui!';

// 3. Insere na página
const body = document.querySelector('body');
body.appendChild(paragrafo);

// 4. Remover um elemento
paragrafo.remove();
```

Document: ler e alterar atributos

`getAttribute / setAttribute / hasAttribute /
removeAttribute`

getAttribute / setAttribute / hasAttribute / removeAttribute

```
const link = document.querySelector('a');

// 1. Ler um atributo
console.log(link.getAttribute('href'));

// 2. Alterar um atributo
link.setAttribute('href', 'https://novo-link.com');

// 3. Verificar se existe um atributo
link.hasAttribute('target'); // true ou false

// 4. Remover atributo
link.removeAttribute('target');
```

fetch e consumo de APIs

...

APIs

- <https://api.nasa.gov/>
- <https://rickandmortyapi.com/>
- <https://www.omdbapi.com/>
- <https://pokeapi.co/>

Bugs e erros

Tratamento de erros e exceções

Eventos

Tratamento de eventos / AddEventHandler

Arrays

Criação e manipulação de Arrays

Módulos

...

Canvas

Desenhando na `<canvas>`

Referências Bibliográficas

EICH, Brendan. **A Brief History of JavaScript** . Disponível em:

<https://www.youtube.com/watch?v=qKJP93dWn40>.

EICH, Brendan. **How to fix the web**. TEDxVienna. Disponível em:

<https://www.youtube.com/watch?v=zlcnOr81IPc>.

ECMA-262 - Ecma International. **Ecma International**. Disponível em:

<https://ecma-international.org/publications-and-standards/standards/ecma-262/>

CROCKFORD, Douglas. **JavaScript: The World's Most Misunderstood Programming Language**.

Disponível em: <https://www.crockford.com/javascript/javascript.html>

GLOBAL STATS. **Most Popular Web Browsers: Rise and Fall (1992-2025)**. Disponível em:

<https://www.youtube.com/watch?v=K9G0rmPh6Wo>.

Referências Bibliográficas

HOFFMANN, Jay. **Chapter 10: Browser Wars** | CSS-Tricks. CSS-Tricks. Disponível em: <https://css-tricks.com/chapter-10-browser-wars/>

MDN - Mozilla Developer Network. **Sobre JavaScript**. Disponível em: <https://developer.mozilla.org/pt-BR/docs/conflicting/Web/JavaScript>.

PETER, Ian. **The dotcom bubble**. Nethistory.info. Disponível em: <https://www.nethistory.info/History of the Internet/browserwars.html>.

SHOZE. **browser wars**. Blogspot.com. Disponível em: <https://shoze.blogspot.com/2011/06/browser-wars.html>.

SWARTZ, Jon. **Microsoft Pulls Prank / Company takes browser war to Netscape's lawn**. SFGATE. Disponível em: <https://www.sfgate.com/business/article/Microsoft-Pulls-Prank-Company-takes-browser-war-2803749.php>